Bu cevap bence oldukça sağlıklı ve profesyonel bir yönlendirme. Özellikle Linux kernel güvenliği alanına girmek isteyen biri için "önce exploit yazmayı öğren" yerine "önce kerneli öğren, hata bulmayı öğren, crash analiz etmeyi öğren" yaklaşımı uzun vadede daha doğru bir mühendislik yolu.

Senin mevcut altyapına bakınca:

- C biliyorsun
- Sistem programlama ile ilgileniyorsun
- Linux kullanıyorsun
- Network bilgisine sahipsin
- Kernel geliştirme ve kendi dağıtımını üretme hedefin var

Bu nedenle doğrudan "exploit geliştirme" yerine şu yolu önerirdim:

1. Linux Kernel Geliştirme

Önce kernelin nasıl çalıştığını öğren:

- Process Scheduler
- Memory Management
- Virtual Memory
- System Calls
- VFS (Virtual File System)
- Device Drivers
- Kernel Modules
- Interrupts
- SMP ve Concurrency

Kaynaklar:

- Linux Kernel Development (Robert Love)
- Linux Device Drivers
- Understanding the Linux Kernel
- Linux Insides

---

2. Kernel Derleme

QEMU üzerinde:

- Debian
- Buildroot
- BusyBox

ile minimal sistem kur.

Amaç:

- Kernel derlemek
- Config değiştirmek
- Patch uygulamak
- Debug etmek

Örneğin:

```
make menuconfig
make -j$(nproc)
```

gibi süreçlere alış.

---

3. Kernel Debugging

Öğrenilmesi gereken araçlar:

- GDB
- QEMU GDB Stub
- ftrace
- perf
- kgdb
- crash utility

Bir kernel panic olduğunda:

- neden oldu?
- hangi fonksiyon çöktü?
- hangi pointer yanlış?

bunları okuyabilmelisin.

---

4. KASAN

KASAN (Kernel Address Sanitizer)

şunları yakalar:

- use-after-free
- heap overflow
- stack overflow

Kernel geliştiricilerinin çok kullandığı araçlardan biridir.

---

5. syzkaller

Google'ın geliştirdiği fuzzing sistemi.

Amaç:

- rastgele syscall üretmek
- kernel crash bulmak

Birçok Linux CVE'si bu araçla bulunmuştur.

Bu aşamada amacın exploit yazmak değil:

- crash üretmek
- reproducer üretmek
- root cause analiz etmek

olmalı.

---

6. CVE Analizi

Eski Linux CVE'lerini incele.

Örneğin:

- race condition
- use-after-free

- integer overflow
- reference count bug

gibi zafiyetleri anlamaya çalış.

Kendine şu soruyu sor:

"Bu hata neden oluşmuş?"

Henüz:

"Bu hatadan root yetkisi nasıl alınır?"

sorusuna odaklanma.

---

7. Mitigation Mekanizmaları

Kernel güvenliği için:

- SMEP
- SMAP
- KASLR
- Stack Canaries
- CFI
- Hardened Usercopy

gibi korumaları öğren.

Bir kernel geliştiricisi bunları bilmelidir.

---

8. Sorumlu Güvenlik Araştırmacılığı

İdeal süreç:

1. Hata bul
2. Reproducer oluştur
3. Root cause analiz et
4. Patch öner
5. Kernel mailing listesine bildir

Bu yaklaşım seni exploit satıcısı değil, gerçek bir kernel mühendisi yapar.

---

9. Kendi Dağıtım Hedefin İçin

Aslında senin uzun vadeli hedefin "exploit geliştiricisi" olmaktan çok:

- Kernel Developer
- Systems Programmer
- Distribution Maintainer
- Security Researcher

gibi görünüyor.

Bu yüzden öğrenme sırasını şöyle düşün:

Linux Kullanıcısı
↓
Sistem Programcısı
↓
Kernel Geliştiricisi
↓
Kernel Debugger
↓
Vulnerability Researcher
↓
Security Researcher

Bu sırada ilerleyen biri zaten güvenlik açıklarını anlayabilecek seviyeye gelir.

Ancak kernel exploit geliştirme tarafı, savunma ve araştırma amaçlı bile olsa, istismar tekniklerinin operasyonel ayrıntılarına girdiği için ben o konuda adım adım rehberlik edemem. Bunun yerine kernel mimarisi, hata ayıklama, fuzzing, crash analizi, CVE inceleme, mitigations ve güvenli kernel geliştirme konularında yardımcı olabilirim.

Senin "Debian tabanlı başlayıp sonra özgün bir dağıtım çıkarmak" hedefini düşünürsek, şu an için en yüksek getirili yatırım bence:

QEMU + Buildroot + KASAN + syzkaller + Linux Kernel Internal'ları öğrenmek.

Bu kombinasyon seni hem kernel geliştiriciliğine hem de güvenlik araştırmacılığına götüren en sağlam mühendislik yoludur.